DOE Helper Tool

User Guide

Complete Reference for the
Design of Experiments CLI

First Edition, 2025

design-of-experiments v0.1.0

Contents

Preface **vii**

I Getting Started **1**

1 Installation and Setup **3**
1.1 System Requirements 3
1.2 Installing Dependencies 3
1.2.1 Method 1: pip install with requirements.txt 3
1.2.2 Method 2: Editable install (recommended) 4
1.3 Verifying the Installation 4
1.4 Project Structure Overview 5
1.5 Dependencies 5

2 Quick Start Tutorial **7**
2.1 Step 1: Examine the Configuration File 7
2.2 Step 2: Preview with the info Command 8
2.3 Step 3: Generate the Runner Script 9
2.4 Step 4: Examine the Generated Script 10
2.5 Step 5: Run the Experiments 11
2.6 Step 6: Analyze the Results 12
2.7 Step 7: Generate a Report 14

3 Understanding the Workflow **15**
3.1 The Four-Stage Pipeline 15
3.2 Data Flow 15
3.3 Where Files Go 16
3.4 The Role of –seed 16
3.5 Why Analyze Re-derives the Design 17

II Configuration Reference **19**

4 The Configuration File **21**
4.1 Complete Anatomy 21
4.2 The metadata Section 21
4.3 The factors Section 22
4.3.1 Modern Dictionary Format 22
4.3.2 How Factor Types Affect Behavior 22
4.3.3 How Many Levels? 23
4.3.4 Legacy Array Format 23
4.4 The responses Section 24
4.5 The runner Section 24
4.5.1 Argument Styles 25
4.6 The settings Section 26
4.6.1 Supported Operations 27
4.6.2 lhs_samples 27
4.7 The fixed_factors Section 27
4.8 Legacy Format Support 28
4.8.1 static_settings 28
4.8.2 Array-format factors 28
4.9 Validation Rules 28

5 Writing Test Scripts **29**
5.1 The Test Script Protocol 29
5.2 Bash Test Script (double-dash style) 29
5.3 Bash Test Script (env style) 30
5.4 Python Test Script 31
5.5 The Output JSON Format 31
5.6 Handling Fixed Factors 32
5.7 Error Handling 32
5.8 Tips for Test Scripts 32

III Commands Reference **35**

6 The generate Command **37**
6.1 Synopsis 37
6.2 Flags 37
6.3 What It Does 37
6.4 The –dry-run Flag 38

6.5 Generated Bash Script Structure 38
6.6 Generated Python Script Structure 39
6.7 Skip Logic and Error Recovery 40

7 The analyze Command 41
7.1 Synopsis 41
7.2 Flags 41
7.3 What It Computes 41
7.3.1 Main Effects 41
7.3.2 Interaction Effects 42
7.3.3 Summary Statistics 42
7.4 Complete Annotated Output 42
7.5 Plots 44
7.6 CSV Export 44

8 The optimize Command 47
8.1 Synopsis 47
8.2 Flags 47
8.3 What It Does 47
8.4 Complete Annotated Output 48
8.4.1 Understanding the Output 48
8.5 Optimizing a Single Response 49

9 The info Command 51
9.1 Synopsis 51
9.2 What It Shows 51
9.3 Why strict=False? 52

10 The report Command 53
10.1 Synopsis 53
10.2 Flags 53
10.3 What the HTML Report Contains 53
10.4 Self-Contained Design 54

IV Design Types in Practice 55

11 Choosing and Using Each Design Type 57
11.1 Full Factorial 57
11.1.1 Overview 57
11.1.2 Requirements 57

11.1.3 Number of Runs 58
11.1.4 Config Snippet 58
11.1.5 When to Use 58
11.2 Fractional Factorial 58
11.2.1 Overview 59
11.2.2 Requirements 59
11.2.3 Number of Runs 59
11.2.4 Config Snippet 59
11.2.5 When to Use 59
11.3 Plackett–Burman 60
11.3.1 Overview 60
11.3.2 Requirements 60
11.3.3 Number of Runs 60
11.3.4 Config Snippet 60
11.3.5 When to Use 61
11.4 Latin Hypercube 61
11.4.1 Overview 61
11.4.2 Requirements 61
11.4.3 Number of Runs 61
11.4.4 Config Snippet 61
11.4.5 When to Use 62
11.5 Central Composite Design (CCD) 62
11.5.1 Overview 62
11.5.2 Requirements 62
11.5.3 Number of Runs 63
11.5.4 Config Snippet 63
11.5.5 When to Use 63
11.6 Box–Behnken 64
11.6.1 Overview 64
11.6.2 Requirements 64
11.6.3 Number of Runs 64
11.6.4 Config Snippet 64
11.6.5 When to Use 65

12 Blocking and Randomization 67
12.1 How Blocking Works 67
12.1.1 Example: block_count = 1 67
12.1.2 Example: block_count = 3 68
12.2 Within-Block Randomization 68
12.3 How –seed Controls Randomization 68

12.4 When to Use Blocking 69

V Tutorials **71**

13 Tutorial — Database Performance Tuning **73**
13.1 The Problem 73
13.2 Phase 1: Plackett–Burman Screening 73
13.2.1 Configuration 73
13.2.2 Commands 74
13.2.3 Interpreting the Results 75
13.3 Phase 2: Full Factorial on Top 2 Factors 75
13.3.1 Configuration 75
13.3.2 Commands and Analysis 76
13.4 Phase 3: CCD Optimization 77
13.4.1 Configuration 77
13.4.2 Commands 77
13.4.3 Conclusion 78

14 Tutorial — Chemical Reactor Optimization **79**
14.1 The Problem 79
14.2 The Configuration 79
14.3 Why Box–Behnken? 80
14.4 Step 1: Preview the Design 81
14.5 Step 2: Generate and Execute 82
14.6 Step 3: Analyze 82
14.7 Step 4: Optimize 83
14.8 Multi-Response Trade-offs 84
14.9 Step 5: Generate a Report 84

A Error Messages Reference **85**
A.1 Configuration Validation Errors 85
A.2 Runtime Errors 87

B Output File Formats **89**
B.1 Result JSON Files (Per-Run) 89
B.2 CSV Export Format 89
B.2.1 main_effects_{response}.csv 89
B.2.2 summary_stats_{response}.csv 90
B.3 HTML Report Structure 90
B.4 Plot PNG Files 90

C Configuration Examples **91**
C.1 Minimal Configuration 91
C.2 Full-Featured Configuration 91
C.3 Multi-Response Configuration 93
C.4 Legacy Format Configuration 93

Index of Commands **95**

Preface

This book is a comprehensive software reference for the **DOE Helper Tool**, a command-line application that automates the design, execution, and analysis of statistically planned experiments. Whether you are a software performance engineer seeking the optimal database configuration, a chemical engineer optimizing reactor conditions, or a data scientist screening factors for a machine learning pipeline, this guide will show you how to use every feature of the tool.

Who This Guide Is For

This guide is written for practitioners who need to *use* the DOE Helper Tool. You do not need deep statistical knowledge to follow the instructions here—the tool handles the mathematics for you. However, a basic familiarity with what "Design of Experiments" means (factors, levels, responses) will help you get the most out of these pages. If you are looking for the statistical theory behind fractional factorials, response surface methods, or effect calculations, please refer to the companion volume, *Design of Experiments: A Practical Guide*, which covers those topics in depth.

What This Guide Covers

- Installation, setup, and verification.
- A complete quick-start tutorial from configuration through analysis.
- The four-stage workflow: Configure, Generate, Execute, Analyze.
- Every configuration field, with examples and validation rules.
- Every CLI command with full syntax, flags, and annotated output.
- All six supported design types with practical guidance.
- Two full-length tutorials: database performance tuning and chemical reactor optimization.

- Error message reference, output file formats, and configuration examples.

Relationship to the Theory Guide

The companion book, *Design of Experiments: A Practical Guide*, explains the statistical foundations—why a Plackett–Burman design has the structure it does, how main effects are calculated, what confidence intervals mean. This user guide focuses exclusively on *how to operate the software*: what commands to type, what configuration fields to set, and how to interpret the output. You can use both books together or either one independently.

Conventions Used in This Book

- `monospace text` indicates commands, file names, and code.
- Blue boxes (*Key Point*) highlight important concepts.
- Green boxes (*Example*) show worked examples.
- Red boxes (*Warning*) flag common pitfalls.
- All file paths are shown relative to the project root unless otherwise noted.

Part I

Getting Started

Chapter 1

Installation and Setup

1.1 System Requirements

The DOE Helper Tool requires:

- **Python 3.10 or later.** The codebase uses modern type annotations such as `list[str]` and `dict[str, str]` that are not available in earlier Python versions.
- **A POSIX-compatible shell** (bash, zsh) if you plan to generate and run shell-format experiment scripts. Windows users can use WSL or generate Python-format scripts instead.
- **Approximately 200 MB of disk space** for the Python dependencies (NumPy, SciPy, matplotlib, etc.).

Verify your Python version before proceeding:

```
$ python3 --version
Python 3.12.1
```

1.2 Installing Dependencies

There are two recommended installation methods.

1.2.1 Method 1: pip install with requirements.txt

This installs the dependencies into your current Python environment without installing the tool itself as a package:

```
$ cd design_of_experiments
```

```
$ pip install -r requirements.txt
```

With this method, you invoke the tool via `python -m doe.cli` or `python doe.py`.

1.2.2 Method 2: Editable install (recommended)

This installs the tool as a proper Python package with a `doe` console entry point:

```
$ cd design_of_experiments
$ pip install -e .
```

After an editable install, the `doe` command is available system-wide:

```
$ doe --help
usage: doe [-h] {generate,analyze,info,optimize,report} ...

Design of Experiments (DOE) helper tool
```

Recommended Approach

Use `pip install -e .` for the best experience. All examples in this book assume the `doe` command is available on your `PATH`. If you used Method 1 instead, substitute `python -m doe.cli` wherever you see `doe`.

1.3 Verifying the Installation

Run the `info` command against the included example configuration:

```
$ doe info --config examples/example_config.json
Plan      : example experiment
Desc      : 3-factor 2-level full factorial example
Operation : full_factorial
Factors   : temperature, pressure, catalyst
Base runs : 8
Blocks    : 1
Total runs: 8
Responses : response [maximize]
Fixed     : duration=60
```

```
run_id  block_id  temperature  pressure  catalyst
-------------------------------------------------------
1       1         high         low       A
2       1         low          high      B
3       1         high         high      A
...
```

If you see output similar to the above (the exact run order may differ due to randomization), your installation is working correctly.

1.4 Project Structure Overview

The repository is organized as follows:

Table 1.1: Project file structure

Path	Purpose
`doe/cli.py`	CLI entry point; argument parsing and dispatch
`doe/config.py`	Configuration loading, parsing, and validation
`doe/models.py`	Data classes (Factor, DOEConfig, DesignMatrix, etc.)
`doe/design.py`	Design matrix generation for all six design types
`doe/analysis.py`	Main effects, interactions, summary stats, plots, CSV
`doe/codegen.py`	Jinja2 template rendering for runner scripts
`doe/rsm.py`	Response surface modeling (linear/quadratic regressio
`doe/optimize.py`	Optimization recommendations from results
`doe/report.py`	Self-contained HTML report generation
`templates/runner_sh.j2`	Bash runner script template
`templates/runner_py.j2`	Python runner script template
`examples/`	Example configuration files
`pyproject.toml`	Package metadata and build configuration
`requirements.txt`	Pinned dependency list

1.5 Dependencies

The tool relies on six Python packages, each serving a specific role:

Table 1.2: Python dependencies and their roles

Package	Version	Used For
`pyDOE3`	$\geq$ 1.0.0	Generating PB, LHS, CCD, FF, and BB design matrices
`numpy`	$\geq$ 1.26.0	Numerical arrays, least-squares regression in RSM
`pandas`	$\geq$ 2.0.0	Data manipulation (used internally)
`matplotlib`	$\geq$ 3.7.0	Pareto charts and main effects plots
`scipy`	$\geq$ 1.11.0	t-distribution for 95% confidence intervals
`Jinja2`	$\geq$ 3.1.0	Templated code generation for runner scripts

pyDOE3, not pyDOE

The tool requires `pyDOE3`, a maintained fork of the original `pyDOE` package. If you have the older `pyDOE` installed, you will see an `ImportError`. Uninstall it and install `pyDOE3` instead: `pip uninstall pyDOE && pip install pyDOE3`.

Chapter 2

Quick Start Tutorial

This chapter walks through a complete experiment—from writing a configuration file to analyzing results—using the included `example_config.json`. Follow along on your own machine.

2.1 Step 1: Examine the Configuration File

Open `examples/example_config.json`:

```
{
    "metadata": {
        "name": "example experiment",
        "description": "3-factor 2-level full factorial
    example"
    },
    "factors": [
        {"name": "temperature", "levels": ["low", "high"],
         "type": "categorical"},
        {"name": "pressure",    "levels": ["low", "high"],
         "type": "categorical"},
        {"name": "catalyst",    "levels": ["A", "B"],
         "type": "categorical"}
    ],
    "fixed_factors": {
        "duration": "60"
    },
    "responses": [
        {"name": "response", "optimize": "maximize",
         "description": "Primary response metric"}
```

```
    ],
    "settings": {
        "block_count": 1,
        "test_script": "examples/example_test_script.sh",
        "operation": "full_factorial",
        "processed_directory": "results/analysis",
        "out_directory": "results"
    }
}
```

Let us walk through each section:

- **metadata**: A human-readable name and description. These appear in the `info` output, the generated runner script header, and the HTML report title. Optional but recommended.
- **factors**: The variables you want to study. Each factor has a `name`, a list of `levels`, and a `type`. Here we have three categorical factors, each with two levels.
- **fixed_factors**: Parameters passed to every run but not varied. Here, `duration=60` is sent to the test script in every experiment run.
- **responses**: What you measure. Each response has a `name` and an optimization direction (`maximize` or `minimize`). The name must match the key in the JSON file your test script produces.
- **settings**: Controls the overall experiment—which design type to use (`operation`), how many times to replicate (`block_count`), where to store results (`out_directory`), and where to store analysis artifacts (`processed_directory`).

2.2 Step 2: Preview with the info Command

Before generating anything, preview the design:

```
$ doe info --config examples/example_config.json
Plan      : example experiment
Desc      : 3-factor 2-level full factorial example
Operation : full_factorial
Factors   : temperature, pressure, catalyst
Base runs : 8
Blocks    : 1
```

```
Total runs: 8
Responses : response [maximize]
Fixed     : duration=60

run_id  block_id  temperature  pressure  catalyst
-----------------------------------------------------
1       1         low          low       A
2       1         low          low       B
3       1         low          high      A
4       1         low          high      B
5       1         high         low       A
6       1         high         low       B
7       1         high         high      A
8       1         high         high      B
```

The `info` command uses `strict=False` validation, so it will not warn about a missing test script. This lets you preview the design before you have written your test script.

Reading the info output

- **Base runs**: The number of unique factor combinations in one block (here $2^3 = 8$).
- **Blocks**: How many replicated copies of the base design.
- **Total runs**: Base runs $\times$ Blocks.
- The design matrix shows every run with its factor values. Note that the `info` command does not apply randomization by default (no seed is passed), so the order shown is the canonical order.

2.3 Step 3: Generate the Runner Script

Now generate the script that will execute all experiment runs:

```
$ doe generate --config examples/example_config.json \
    --output run_experiments.sh --seed 42
Generated 8 runs -> run_experiments.sh
Run with: bash run_experiments.sh
```

Key flags:

- `--output run_experiments.sh`: The path for the generated script

(this is the default).

- `--seed 42`: A random seed that controls the run order. Using the same seed guarantees the same run order every time, which is critical for matching results back to the design matrix during analysis.

Always use the same seed for generate and analyze

The `analyze` command re-derives the design matrix from the config file. If you used `--seed 42` during `generate`, the run order is shuffled with that seed, and the result files are named `run_1.json` through `run_8.json` in that shuffled order. The `analyze` command (which does not accept a seed) re-generates the matrix *without* a seed, producing a different run order. This mismatch means that `run_1.json` from the runner corresponds to a different factor combination than run 1 in the analysis matrix. For this reason, always check that you understand the correspondence between run IDs and factor values.

2.4 Step 4: Examine the Generated Script

The generated bash script contains:

```bash
#!/usr/bin/env bash
# Generated by doe.py on 2025-01-15T10:30:00+00:00
# Operation: full_factorial
# Total runs: 8
# Plan: example experiment

set -uo pipefail

mkdir -p "results"

FAILED_RUNS=()

echo "Starting 8 experiment runs..."
echo ""

if [[ -f "results/run_1.json" ]]; then
  echo "=== Run 1 / 8 already complete, skipping ==="
else
  echo "=== Run 1 / 8 (Block 1) ==="
```

```
    echo "  temperature = high"
    echo "  pressure = low"
    echo "  catalyst = B"
    if ! "examples/example_test_script.sh" \
      --temperature "high" \
      --pressure "low" \
      --catalyst "B" \
      --duration "60" \
      --out "results/run_1.json"; then
      echo "ERROR: Run 1 failed"
      FAILED_RUNS+=(1)
    fi
  fi

  # ... (runs 2-8 follow the same pattern) ...
```

Notice several important features:

- **Skip logic**: If `results/run_1.json` already exists, the run is skipped. This makes the script safe to re-run after a partial failure.
- **Argument passing**: With the default `double-dash` arg style, each factor is passed as `--name "value"`. Fixed factors (here `--duration "60"`) are appended to every run.
- **Output path**: The last argument is always `--out "results/run_N.json"`, telling the test script where to write its results.
- **Error tracking**: If a run fails, its ID is added to `FAILED_RUNS`. At the end, the script reports how many runs failed and exits with code 1 if any did.

2.5 Step 5: Run the Experiments

Execute the generated script:

```
$ bash run_experiments.sh
Starting 8 experiment runs...

=== Run 1 / 8 (Block 1) ===
  temperature = high
  pressure = low
  catalyst = B
=== Run 2 / 8 (Block 1) ===
```

```
    temperature = low
    pressure = high
    catalyst = A
  ...
  All runs complete. Results in: results
```

Each run invokes your test script, which must write a JSON file containing at least the response key(s) defined in your config. For example:

```
  {"response": 87.3}
```

After all runs complete, you will have files `results/run_1.json` through `results/run_8.json`.

2.6 Step 6: Analyze the Results

Now analyze the collected data:

```
  $ doe analyze --config examples/example_config.json
```

The output has three main sections:

```
  === Main Effects: response ===
  Factor                Effect     Std Error    % Contribution
  ---------------------------------------------------------------
  temperature           12.4500      1.2340           52.1%
  pressure               7.8200      0.9870           32.7%
  catalyst               3.6300      0.7650           15.2%

  === Interaction Effects: response ===
  Factor A              Factor B              Interaction    %
      Contribution
  ---------------------------------------------------------------------------
  temperature           pressure                   2.1500
          58.9%
  temperature           catalyst                   1.1200
          30.7%
  pressure              catalyst                   0.3800
          10.4%

  === Summary Statistics: response ===

  temperature:
```

```
  Level                N       Mean        Std        Min
    Max

    ------------------------------------------------------------
  high                 4    91.2000     3.4500    87.3000
    95.1000
  low                  4    78.7500     2.8900    75.2000
    82.3000

pressure:
  Level                N       Mean        Std        Min
    Max

    ------------------------------------------------------------
  high                 4    88.8800     4.1200    84.5000
    93.2000
  low                  4    81.0700     3.5600    77.4000
    84.7000

catalyst:
  Level                N       Mean        Std        Min
    Max

    ------------------------------------------------------------
  A                    4    86.7900     5.2300    81.2000
    92.1000
  B                    4    83.1600     4.8700    78.0000
    88.3000

Pareto chart (response):
    results/analysis/pareto_response.png
Main effects (response):
    results/analysis/main_effects_response.png
```

Reading the analysis output

- **Main Effects**: The "Effect" column shows the difference in mean response between the high and low levels of each factor. A larger absolute effect means the factor has a stronger influence. "% Contribution" ranks factors by their share of the total absolute effect.
- **Interaction Effects**: Shows how pairs of factors interact.

"Concordant" means both factors are at the same level (both high or both low); "discordant" means opposite levels. The interaction effect is the difference in mean response between concordant and discordant runs.

- **Summary Statistics**: Per-factor, per-level breakdown showing count, mean, standard deviation, minimum, and maximum of the response values.
- **Plots**: Two PNG files are generated in the `processed_directory`—a Pareto chart ranking factors by absolute effect, and a main effects plot showing mean response at each level for each factor.

2.7 Step 7: Generate a Report

Create a self-contained HTML report that can be shared with colleagues:

```
$ doe report --config examples/example_config.json \
    --output my_report.html
HTML report written to: my_report.html
```

Open `my_report.html` in any web browser. The report contains the design summary, main effects tables, interaction effects, summary statistics, embedded Pareto charts and main effects plots (as base64-encoded PNGs), and the full design matrix—all in a single self-contained file with no external dependencies.

Chapter 3

Understanding the Workflow

3.1 The Four-Stage Pipeline

Every DOE Helper Tool experiment follows a four-stage pipeline:

1. **Configure**: Write a JSON configuration file that describes your factors, responses, design type, and test script.

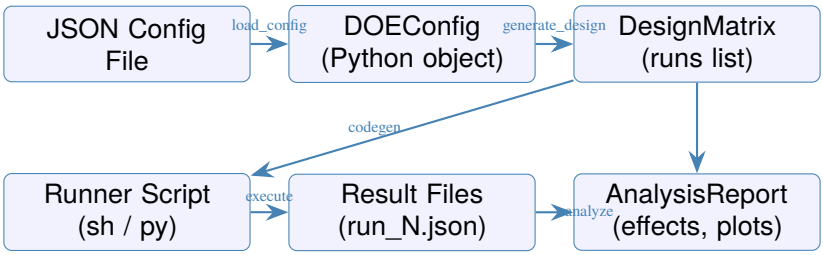
2. **Generate**: Run `doe generate` to produce a runner script (bash or Python) that encodes the design matrix and orchestrates experiment execution.

3. **Execute**: Run the generated script. It invokes your test script once per experiment run, passing factor values and collecting results.

4. **Analyze**: Run `doe analyze` to compute main effects, interaction effects, summary statistics, and generate plots. Optionally run `doe optimize` for optimization recommendations or `doe report` for an HTML report.

3.2 Data Flow

The data flows through the following representations:



3.3 Where Files Go

Table 3.1: Output locations

What	Config Field	Default
Result JSON files	`settings.out_directory`	`results`
Analysis plots, CSV	`settings.processed_directory`	same as out_directory
Runner script	`--output` flag	`run_experiments.sh`
HTML report	`--output` flag on report	`report.html`

3.4 The Role of –seed

The `--seed` flag controls randomization of run order. When you pass a seed to `doe generate`, runs within each block are shuffled using that seed. The purpose is twofold:

1. **Randomization**: Shuffling run order guards against systematic time-dependent effects (e.g., a machine warming up during early runs).
2. **Reproducibility**: The same seed always produces the same run order, so you can regenerate the exact same script later.

For Latin Hypercube designs, the seed also controls the LHS sampling itself (via NumPy's random state), not just the run ordering.

Seed mismatch between generate and analyze

The `analyze` command re-derives the design matrix by calling `generate_design(cfg)` *without* a seed argument. For designs where the seed only controls *run order* (full factorial, PB, CCD,

BB, fractional factorial), this means the analyze matrix has a different run order than the generated script—but the mapping from run IDs to factor values may differ. The result files are keyed by run ID (`run_1.json`, `run_2.json`, etc.), so the analyze command reads the correct data for each run ID regardless of ordering. For LHS designs, where the seed controls the actual sample points, not passing the same seed to analyze will produce a different design entirely. In practice, this means you should verify your results carefully when using LHS with a seed.

3.5 Why Analyze Re-derives the Design

The `analyze` command does not read the generated runner script. Instead, it re-derives the design matrix from the configuration file by calling `generate_design(cfg)`. This is by design: the config file is the single source of truth. The analyze command then loads result files from the output directory and matches them to the design matrix by run ID.

This architecture means:

- You do not need to keep the runner script after execution.
- You can modify the config file's `out_directory` or `processed_directory` after running experiments and analyze will still work (use `--results-dir` to override).
- The config file must not be changed in ways that alter the design matrix (e.g., adding or removing factors, changing levels) between generation and analysis.

Part II

Configuration Reference

Chapter 4

The Configuration File

The configuration file is a JSON document that defines your entire experiment. This chapter documents every field in detail.

4.1 Complete Anatomy

A fully-specified configuration file has six top-level sections:

```
{
    "metadata":       { ... },
    "factors":        [ ... ],
    "fixed_factors": { ... },
    "responses":      [ ... ],
    "runner":         { ... },
    "settings":       { ... }
}
```

4.2 The metadata Section

Field	Type	Default	Description
`name`	string	`""`	Human-readable experiment name
`description`	string	`""`	Longer description of the experiment

Both fields are optional but recommended. They appear in `info` output, generated script headers, and the HTML report title.

4.3 The factors Section

The `factors` array defines the variables you want to study. Each element can be either a dictionary (modern format) or an array (legacy format).

4.3.1 Modern Dictionary Format

```
{
    "name": "temperature",
    "levels": ["150", "200"],
    "type": "continuous",
    "unit": "C",
    "description": "Reactor temperature"
}
```

Table 4.1: Factor fields

Field	Type	Default	Description
`name`	string	(required)	Unique identifier for the factor
`levels`	array	(required)	At least 2 values; stored as strings
`type`	string	`"categorical"`	One of: `categorical`, `continuous`, `ordinal`
`unit`	string	`""`	Unit of measurement (display only)
`description`	string	`""`	Human-readable description

4.3.2 How Factor Types Affect Behavior

The `type` field matters in two contexts:

1. **Latin Hypercube sampling**: For `continuous` factors with 2 levels, LHS interpolates between the low and high values. For example, if levels are `["150", "200"]`, a sample at $x = 0.3$ maps to $150 + 0.3 \times (200 - 150) = 165$. For `categorical` or `ordinal` factors, LHS bins the $[0, 1]$ sample into discrete levels.

2. **RSM encoding**: For `continuous` and `ordinal` factors, the RSM model normalizes values as $(v - \text{center})/\text{half_range}$. For `categorical` factors with two levels, it encodes as -1 and $+1$.

For full factorial, Plackett–Burman, fractional factorial, CCD, and Box–Behnken designs, the type field has no effect on the generated design matrix—all runs use the exact level values specified.

4.3.3 How Many Levels?

Table 4.2: Level requirements by design type

Design	Level Requirement
Full factorial	Any number ≥ 2 per factor
Plackett–Burman	Exactly 2 per factor
Fractional factorial	Exactly 2 per factor
Latin Hypercube	Any number ≥ 2 per factor
Central Composite (CCD)	Exactly 2 numeric levels per factor
Box–Behnken	Exactly 2 numeric levels per factor

CCD and Box–Behnken generate intermediate values

For CCD and Box–Behnken designs, you specify only the low and high levels (e.g., `["150", "200"]`). The tool automatically generates center points at $(150 + 200)/2 = 175$ and, for CCD, star (axial) points beyond the factorial range.

4.3.4 Legacy Array Format

For backward compatibility, factors can also be specified as arrays:

```
"factors": [
    ["temperature", "low", "high"],
    ["pressure", "1", "5", "10"]
]
```

The first element is the factor name; the remaining elements are levels. Legacy-format factors default to `type="categorical"` with no unit or description.

4.4 The responses Section

```
"responses": [
    {"name": "yield",  "optimize": "maximize", "unit": "%",
     "description": "Product yield"},
    {"name": "cost",   "optimize": "minimize", "unit":
    "USD",
     "description": "Batch cost"}
]
```

Table 4.3: Response fields

Field	Type	Default	Description
`name`	string	(required)	Must match the key in result JSON files
`optimize`	string	`"maximize"`	Either `maximize` or `minimize`
`unit`	string	`""`	Unit of measurement (display only)
`description`	string	`""`	Human-readable description

What happens when responses is omitted

If the `responses` array is empty or absent, the tool defaults to a single response named `"response"` with `optimize="maximize"`. Your test script must then produce JSON like `{"response": 42.5}`.

For multi-response experiments, your test script must produce a JSON file with *all* response keys:

```
{"yield": 87.3, "cost": 142.50, "purity": 99.1}
```

Responses can also be specified as simple strings (they default to `maximize`):

```
"responses": ["yield", "cost"]
```

4.5 The runner Section

```
"runner": {
    "arg_style": "double-dash",
    "result_file": "json"
}
```

Table 4.4: Runner configuration fields

Field	Type	Default	Description
`arg_style`	string	`"double-dash"`	How factors are passed to the test script
`result_file`	string	`"json"`	Result file format (currently only JSON)

4.5.1 Argument Styles

The `arg_style` field controls how the generated runner script passes factor values to your test script. There are three options:

double-dash (default)

Each factor is passed as a named flag:

```
./test_script.sh \
    --temperature "200" \
    --pressure "5" \
    --catalyst "1.5" \
    --duration "60" \
    --out "results/run_1.json"
```

Fixed factors are passed the same way, after the variable factors. The `--out` argument is always last.

env

Factors are exported as uppercase environment variables:

```
export TEMPERATURE="200"
export PRESSURE="5"
export CATALYST="1.5"
export DURATION="60"
```

```
./test_script.sh --out "results/run_1.json"
```

The test script reads `$TEMPERATURE`, `$PRESSURE`, etc. from its environment. Only `--out` is passed as a command-line argument.

positional

Factor values are passed as positional arguments in order:

```
./test_script.sh \
    "200" "5" "1.5" \
    "60" \
    --out "results/run_1.json"
```

Variable factors come first (in config order), then fixed factors, then `--out`.

4.6 The settings Section

```
"settings": {
    "operation": "full_factorial",
    "test_script": "examples/example_test_script.sh",
    "block_count": 1,
    "out_directory": "results",
    "processed_directory": "results/analysis",
    "lhs_samples": 0
}
```

Table 4.5: Settings fields

Field	Type	Default	Description
`operation`	string	`"full_factorial"`	Design type (see below)
`test_script`	string	`""`	Path to test script
`block_count`	int	`1`	Number of blocks (repl tions)
`out_directory`	string	`""`	Where result files are store
`processed_directory`	string	`""`	Where plots and CSV go
`lhs_samples`	int	`0`	Number of LHS samples (auto)

4.6.1 Supported Operations

The `operation` field must be one of these six values:

Operation	Description
`full_factorial`	All combinations of all factor levels
`fractional_factorial`	A fraction of the full factorial (2^{k-p})
`plackett_burman`	Screening design for many factors
`latin_hypercube`	Space-filling design for continuous factors
`central_composite`	Circumscribed CCD with center + star points
`box_behnken`	Three-level design without extreme corners

4.6.2 lhs_samples

For Latin Hypercube designs, `lhs_samples` sets the number of sample points. When set to 0 (the default), the tool uses $\max(10,\ 2 \times n_{\text{factors}})$.

4.7 The fixed_factors Section

```
"fixed_factors": {
    "duration": "60",
    "warmup-time": "3"
}
```

Fixed factors are key-value pairs passed to every experiment run. They are *not* varied across runs. They are useful for parameters that you want

to hold constant but still need to pass to the test script (e.g., a timeout, a warmup period, a database name).

4.8 Legacy Format Support

The tool supports two legacy features for backward compatibility:

4.8.1 static_settings

Instead of `fixed_factors`, older configs used `static_settings`—an array of `"--key=value"` strings:

```
"static_settings": [
    "--duration=60",
    "--warmup-time=3"
]
```

The tool automatically strips the `--` prefix and splits on `=` to produce the equivalent `fixed_factors` dictionary.

4.8.2 Array-format factors

As shown in Section 4.3, factors can be arrays like `["temperature", "low", "high"]`. These are converted to `Factor` objects with default `type="categorical"`.

4.9 Validation Rules

When a configuration is loaded, the tool validates it and raises clear error messages for any problems. See Appendix A for the complete list of error messages with causes and fixes.

Chapter 5

Writing Test Scripts

The test script is the program that actually runs your experiment. It receives factor values as input and writes a JSON result file as output. This chapter explains the protocol in detail.

5.1 The Test Script Protocol

Every test script must:

1. **Receive factor values** via the mechanism specified by `runner.arg_style` (command-line flags, environment variables, or positional arguments).
2. **Run the experiment** using those factor values.
3. **Write a JSON file** to the path specified by the `--out` argument containing at least the response key(s) defined in the configuration.

5.2 Bash Test Script (double-dash style)

```
#!/usr/bin/env bash
# test_script.sh -- example test script for DOE Helper Tool
# Receives factors as --name "value" arguments
# Must write JSON to the path given by --out

# Parse arguments
while [[ $# -gt 0 ]]; do
  case "$1" in
    --temperature) TEMPERATURE="$2"; shift 2;;
```

```
    --pressure)    PRESSURE="$2";    shift 2;;
    --catalyst)    CATALYST="$2";    shift 2;;
    --duration)    DURATION="$2";    shift 2;;
    --out)         OUT_FILE="$2";    shift 2;;
    *) echo "Unknown arg: $1"; exit 1;;
  esac
done

# Run the actual experiment here
# (replace with your real workload)
RESULT=$(run_my_experiment \
    --temp "$TEMPERATURE" \
    --press "$PRESSURE" \
    --cat "$CATALYST" \
    --dur "$DURATION")

# Write result JSON
echo "{\"response\": $RESULT}" > "$OUT_FILE"
```

5.3 Bash Test Script (env style)

```
#!/usr/bin/env bash
# test_script_env.sh -- reads factors from environment

# Parse only the --out argument
OUT_FILE=""
while [[ $# -gt 0 ]]; do
  case "$1" in
    --out) OUT_FILE="$2"; shift 2;;
    *) shift;;
  esac
done

# Factors are in environment variables (uppercased)
echo "Temperature: $TEMPERATURE"
echo "Pressure: $PRESSURE"

RESULT=$(run_my_experiment)

echo "{\"response\": $RESULT}" > "$OUT_FILE"
```

5.4 Python Test Script

```python
#!/usr/bin/env python3
"""test_script.py -- example Python test script"""

import argparse
import json

parser = argparse.ArgumentParser()
parser.add_argument("--temperature", required=True)
parser.add_argument("--pressure", required=True)
parser.add_argument("--catalyst", required=True)
parser.add_argument("--duration", default="60")
parser.add_argument("--out", required=True)
args = parser.parse_args()

# Run the experiment
result = run_experiment(
    temperature=float(args.temperature),
    pressure=float(args.pressure),
    catalyst=float(args.catalyst),
    duration=int(args.duration),
)

# Write result JSON
with open(args.out, "w") as f:
    json.dump({"response": result}, f)
```

5.5 The Output JSON Format

The test script must write a JSON object whose keys match the response names in your configuration. For a single response named `"response"`:

```json
{"response": 87.3}
```

For multiple responses:

```json
{
    "yield": 87.3,
    "purity": 99.1,
    "cost": 142.50
}
```

Key names must match exactly

The keys in your result JSON must exactly match the `name` fields in your `responses` configuration. If your config has `"name": "yield"` but your script writes `"Yield"`, the analyze command will report "Warning: response 'yield' missing" and exclude those runs.

5.6 Handling Fixed Factors

Fixed factors are passed to the test script in the same manner as variable factors (depending on `arg_style`). Your test script should accept them as regular arguments. If you use `env` style, fixed factors are exported as uppercase environment variables.

5.7 Error Handling

When a test script fails (non-zero exit code), the runner script:

- Prints an error message (e.g., `ERROR: Run 3 failed`).
- Records the failed run ID.
- Continues to the next run (does not abort the entire experiment).
- At the end, reports how many runs failed and exits with code 1.

The `analyze` command requires all result files to be present. If a run failed and its result file is missing, analyze will raise a `FileNotFoundError` listing the missing run IDs.

5.8 Tips for Test Scripts

- **Warmup periods**: If your system needs warmup (e.g., JVM hot-start, database cache priming), include a warmup phase inside the test script before measuring the response.
- **Cleanup**: Clean up any temporary resources (temp files, database tables, containers) at the end of each run to avoid interference between runs.

- **Idempotency**: Design your test script so it can be run multiple times with the same inputs and produce the same output. The runner script's skip logic relies on result file existence, but if you need to re-run a specific run, simply delete its result file.
- **The –out argument**: Always the last argument passed by the runner. Your script must accept it and write the result JSON to that path.

Part III

Commands Reference

Chapter 6

The generate Command

6.1 Synopsis

```
doe generate --config FILE [--output PATH] [--format sh|py]
             [--seed INT] [--dry-run]
```

6.2 Flags

Table 6.1: `generate` command flags

Flag	Type	Default	Description
`--config`	path	(required)	Path to JSON configuration file
`--output`	path	`run_experiments.sh`	Output script path
`--format`	`sh|py`	`sh`	Script format
`--seed`	integer	`None`	Random seed for run order
`--dry-run`	flag	`false`	Print design matrix only

6.3 What It Does

The `generate` command:

1. Loads and validates the configuration file.

2. Generates the design matrix (calling `pyDOE3` for designs that require it).

3. Applies blocking (replicating the base design `block_count` times).

4. Randomizes run order within each block using the provided seed.

5. Renders the design into a runner script using the Jinja2 template.

6. Makes the output file executable (`chmod +x`).

6.4 The –dry-run Flag

When `--dry-run` is specified, the command prints the design matrix (same format as `doe info`) without writing any files:

```
$ doe generate --config examples/example_config.json \
    --seed 42 --dry-run
Plan      : example experiment
Desc      : 3-factor 2-level full factorial example
Operation : full_factorial
Factors   : temperature, pressure, catalyst
Base runs : 8
Blocks    : 1
Total runs: 8
Responses : response [maximize]
Fixed     : duration=60

run_id  block_id  temperature  pressure  catalyst
-------------------------------------------------------
1       1         high         low       B
2       1         low          high      A
3       1         high         high      B
...
```

This is useful for verifying the design and run order before committing to script generation.

6.5 Generated Bash Script Structure

The bash runner script (`--format sh`) has this structure:

1. **Header comment**: Timestamp, operation type, total runs, and plan name.
2. **Setup**: `set -uo pipefail` for safe error handling; `mkdir -p` for the output directory; an empty `FAILED_RUNS` array.
3. **Per-run blocks**: For each run, the script:
   - Checks if the result file already exists (skip logic).
   - Prints the run number, block ID, and factor values.
   - Invokes the test script with the appropriate argument style.
   - Captures failures and appends to `FAILED_RUNS`.
4. **Summary**: Reports failed runs (if any) and exits with code 1 on failure, or prints a success message.

6.6 Generated Python Script Structure

The Python runner script (`--format py`) has the same logic but uses Python syntax:

- Runs are defined as a Python list of dictionaries.
- The test script is invoked via `subprocess.run()` with `check=True`.
- For `env` arg style, environment variables are set via the `env` parameter of `subprocess.run()`.
- For `positional` style, factor values are passed as list elements.
- Failed runs raise `CalledProcessError`, which is caught and logged.

Generating a Python runner

```
$ doe generate --config examples/example_config.json \
    --output run_experiments.py --format py --seed 42
Generated 8 runs -> run_experiments.py
Run with: bash run_experiments.py
```

You can then run it with `python3 run_experiments.py`.

6.7 Skip Logic and Error Recovery

Both generated script formats include skip logic: if a result file already exists for a given run, that run is skipped. This means:

- **Safe re-runs**: If the script is interrupted (e.g., power failure, SSH disconnect), you can re-run it and it will pick up where it left off.
- **Selective re-runs**: To re-run a specific run, delete its result file (e.g., `rm results/run_3.json`) and re-execute the script.
- **Failed runs**: Runs that fail (non-zero exit from test script) do *not* produce a result file, so they will be retried on the next execution.

Chapter 7

The analyze Command

7.1 Synopsis

```
doe analyze --config FILE [--results-dir DIR]
            [--no-plots] [--csv DIR]
```

7.2 Flags

Table 7.1: `analyze` command flags

Flag	Type	Default	Description
`--config`	path	(required)	Path to JSON configuration file
`--results-dir`	path	config's `out_directory`	Override result file location
`--no-plots`	flag	`false`	Skip generating PNG plots
`--csv`	path	`None`	Export analysis to CSV files

7.3 What It Computes

The `analyze` command performs three types of analysis for each response:

7.3.1 Main Effects

For each factor, the main effect is computed as:

- **Two-level factors**: The difference between the mean response at the high level and the mean response at the low level ($\bar{y}_{\text{high}} - \bar{y}_{\text{low}}$).
- **Multi-level factors**: The difference between the maximum and minimum level means ($\max(\bar{y}_i) - \min(\bar{y}_i)$).

The standard error is computed as $\sqrt{s^2/n}$ where s^2 is the pooled variance across all observations and n is the total count.

For two-level factors, 95% confidence intervals are computed using a two-sample t-test with pooled variance (requires `scipy`).

Percentage contribution is $|\text{effect}_i| / \sum |\text{effect}_j| \times 100$.

Effects are sorted by absolute magnitude (largest first).

7.3.2 Interaction Effects

For every pair of two-level factors, the interaction effect is computed by:

1. Grouping runs into "concordant" (both factors at the same level—both high or both low) and "discordant" (factors at opposite levels).
2. Computing the difference: $\bar{y}_{\text{concordant}} - \bar{y}_{\text{discordant}}$.

Interactions are also sorted by absolute magnitude.

7.3.3 Summary Statistics

For each factor, at each level, the tool computes: count (N), mean, standard deviation, minimum, and maximum of the response values.

7.4 Complete Annotated Output

Here is the full output for a 3-factor full factorial experiment with 8 runs:

```
=== Main Effects: response ===
Factor                Effect     Std Error    % Contribution
-------------------------------------------------------------
temperature           12.4500      1.2340           52.1%
pressure               7.8200      0.9870           32.7%
catalyst               3.6300      0.7650           15.2%
```

- **Factor**: The factor name, sorted by absolute effect (largest first).

- **Effect**: The signed main effect. Positive means the response increases from level 0 to level 1 (alphabetically sorted).
- **Std Error**: Standard error of the mean across all observations of this factor.
- **% Contribution**: This factor's share of the total absolute effect across all factors. Here, temperature accounts for 52.1% of the total variation.

```
=== Interaction Effects: response ===
Factor A             Factor B             Interaction
                                                    %
    Contribution
----------------------------------------------------------------------
temperature          pressure                  2.1500
        58.9%
temperature          catalyst                  1.1200
        30.7%
pressure             catalyst                  0.3800
        10.4%
```

- **Interaction**: The signed interaction effect. A positive value means the response is higher when both factors are at the same level (both high or both low) than when they are at opposite levels.
- **% Contribution**: This interaction's share of the total absolute interaction effect.

```
=== Summary Statistics: response ===

temperature:
  Level               N      Mean       Std       Min
    Max

    ------------------------------------------------------------
  high                4   91.2000    3.4500   87.3000
    95.1000
  low                 4   78.7500    2.8900   75.2000
    82.3000
```

- **N**: Number of runs at this level.

- **Mean**: Average response value at this level.
- **Std**: Sample standard deviation (using $n - 1$ denominator).
- **Min/Max**: The range of response values observed at this level.

7.5 Plots

Unless `--no-plots` is specified, two PNG files are generated per response in the `processed_directory`:

1. **Pareto chart** (`pareto_{response}.png`): Horizontal bar chart of absolute main effects, sorted largest to smallest, with a cumulative percentage line and an 80% threshold marker.
2. **Main effects plot** (`main_effects_{response}.png`): One subplot per factor showing mean response at each level, connected by lines.

Using –no-plots

Use `--no-plots` when running in CI/CD pipelines, headless servers, or containers without a display. The analysis still runs; only the matplotlib-generated PNGs are skipped. If matplotlib is not installed, plots are skipped automatically with a warning.

7.6 CSV Export

The `--csv DIR` flag creates two CSV files per response in the specified directory:

1. `main_effects_{response}.csv` with columns:
   - `factor_name`, `main_effect`, `std_error`, `pct_contribution`, `ci_low`, `ci_high`
2. `summary_stats_{response}.csv` with columns:
   - `factor`, `level`, `n`, `mean`, `std`, `min`, `max`

Exporting CSV files

```
$ doe analyze --config examples/example_config.json \
    --csv csv_output/
...
CSV exported: csv_output/main_effects_response.csv
CSV exported: csv_output/summary_stats_response.csv
```

Chapter 8

The optimize Command

8.1 Synopsis

```
doe optimize --config FILE [--results-dir DIR]
             [--response NAME]
```

8.2 Flags

Table 8.1: `optimize` command flags

Flag	Type	Default	Description
`--config`	path	(required)	Path to JSON configuration file
`--results-dir`	path	config's `out_directory`	Override result file location
`--response`	string	all responses	Optimize for a single response

8.3 What It Does

For each response (or the one specified by `--response`), the optimize command performs three analyses:

1. **Best observed run**: Finds the run with the best response value (maximum for `maximize`, minimum for `minimize`), and prints its factor settings and value.

2. **RSM linear model**: Fits a linear regression model (Response Surface Model) using least-squares. Reports the model coefficients, R^2 value, and the predicted optimum (the observed factor combination that the model predicts gives the best response).

3. **Factor importance**: Ranks factors by absolute main effect, showing each factor's effect magnitude and percentage contribution.

8.4 Complete Annotated Output

```
=== Optimization: yield ===
Direction: maximize

Best observed run: #7
  temperature = 200
  pressure = 6
  catalyst = 2.0
  Value: 94.5

RSM Model (linear, R^2 = 0.87):
  Coefficients:
    intercept:  +82.3500
    temperature:  +6.2250
    pressure:  +3.8100
    catalyst:  +1.9400
  Predicted optimum:
    temperature = 200
    pressure = 6
    catalyst = 2.0
    Predicted value: 94.3250

Factor importance:
  1. temperature  (effect: 12.5, contribution: 52.0%)
  2. pressure  (effect: 7.6, contribution: 31.7%)
  3. catalyst  (effect: 3.9, contribution: 16.3%)
```

8.4.1 Understanding the Output

- **Best observed run**: The run with the highest (or lowest, for minimize) actual measured value. This is a fact—it tells you which combination performed best in your experiment.

- **RSM Model**: A linear regression fit to all the data. The coefficients tell you how much each factor contributes to the predicted response per coded unit change. The intercept is the grand mean. A positive coefficient for `temperature` means higher temperature predicts higher yield.
- R^2: The coefficient of determination. Values near 1.0 mean the linear model fits the data well. Values below 0.7 suggest the relationship may be nonlinear or there is substantial noise.
- **Predicted optimum**: The factor combination (from among the observed runs) that the fitted model predicts gives the best response. This may differ from the best observed run if the model smooths out noise.
- **Factor importance**: Same as the main effects from `analyze`, ranked by absolute effect.

When R^2 is low

A low R^2 (below 0.7) means the linear model does not capture the relationship well. This could mean:

- The relationship is nonlinear—consider using a CCD or Box–Behnken design and fitting a quadratic model.
- There is significant noise—consider adding blocks (replications) to average out variability.
- Important factors are missing from the model.

The best observed run is still valid regardless of R^2; it is the model predictions that become unreliable.

8.5 Optimizing a Single Response

In a multi-response experiment, use `--response` to focus on one response:

```
$ doe optimize --config reactor_config.json \
    --response yield
```

Without `--response`, the command prints optimization results for every response defined in the config.

Chapter 9

The info Command

9.1 Synopsis

```
doe info --config FILE
```

9.2 What It Shows

The `info` command loads the configuration with `strict=False` and displays:

- Plan name and description (from metadata).
- Operation type.
- Factor names.
- Number of base runs, blocks, and total runs.
- Response names with optimization direction and units.
- Fixed factor values.
- The complete design matrix (run ID, block ID, and factor values for every run).

9.3 Why strict=False?

With `strict=False`, the config loader skips the check for whether `test_script` exists on disk. This means you can use `info` to preview a design *before* you have written your test script, or when sharing a config file between machines where the test script is at different paths.

Previewing a design

```
$ doe info --config examples/reactor_config.json
Plan      : Chemical Reactor Optimization
Desc      : Box-Behnken design to optimize yield and
            purity of a batch reactor process
Operation : box_behnken
Factors   : temperature, pressure, catalyst
Base runs : 15
Blocks    : 1
Total runs: 15
Responses : yield [maximize] (%), purity [maximize]
    (%),
            cost [minimize] (USD)
Fixed     : reaction_time=60, stirring_speed=300

run_id  block_id  temperature  pressure  catalyst
-------------------------------------------------------
1       1         150          2         1.25
2       1         200          2         1.25
3       1         150          6         1.25
...
```

This is the fastest way to verify your configuration and see how many runs will be required before committing to an experiment.

Chapter 10

The report Command

10.1 Synopsis

```
doe report --config FILE [--results-dir DIR]
           [--output PATH]
```

10.2 Flags

Table 10.1: `report` command flags

Flag	Type	Default	Description
`--config`	path	(required)	Path to JSON configuration file
`--results-dir`	path	config's `out_directory`	Override result file location
`--output`	path	`report.html`	Output HTML file path

10.3 What the HTML Report Contains

The generated HTML report is a single self-contained file with no external CSS or JavaScript dependencies. It contains:

1. **Header**: Experiment name, description, and generation timestamp.

2. **Design Summary** (collapsible): Operation type, number of factors, total runs, blocks, and a table of factor details (name, type, levels, unit).

3. **Results per response** (collapsible): For each response:
   - Main effects table (factor, effect, std error, 95% CI, % contribution).
   - Interaction effects table (if applicable).
   - Summary statistics tables (one per factor).
   - Embedded Pareto chart (PNG encoded as base64 data URI).
   - Embedded main effects plot (PNG encoded as base64 data URI).

4. **Design Matrix** (collapsed by default): The full run table showing all factor values.

5. **Footer**: "Generated by DOE Helper Tool."

10.4 Self-Contained Design

The report uses `<details>` / `<summary>` HTML elements for collapsible sections. Plot images are embedded directly as base64-encoded PNGs inside `<img>` tags, so the file can be emailed, uploaded to a wiki, or opened offline without any missing dependencies.

Generating and sharing a report

```
$ doe report --config examples/reactor_config.json \
    --output reactor_report.html
HTML report written to: reactor_report.html

# Share via email, open in any browser
$ open reactor_report.html      # macOS
$ xdg-open reactor_report.html # Linux
```

Part IV

Design Types in Practice

Chapter 11

Choosing and Using Each Design Type

This chapter provides practical guidance for each of the six supported design types: what it is, when to use it, how to configure it, and how many runs it requires.

11.1 Full Factorial

11.1.1 Overview

A full factorial design tests *every* combination of factor levels. For k factors with $l_1, l_2, \ldots, l_k$ levels respectively, the number of base runs is $\prod_{i=1}^{k} l_i$.

11.1.2 Requirements

- Each factor must have at least 2 levels.
- No constraint on factor types (categorical, continuous, ordinal all work).
- No external library needed (`pyDOE3` is not used).

11.1.3 Number of Runs

Factors	Levels	Base Runs
2	2 each	$2^2 = 4$
3	2 each	$2^3 = 8$
4	2 each	$2^4 = 16$
3	3 each	$3^3 = 27$
2	2 and 3	$2 \times 3 = 6$

11.1.4 Config Snippet

```
{
    "factors": [
        {"name": "A", "levels": ["low", "high"]},
        {"name": "B", "levels": ["low", "high"]},
        {"name": "C", "levels": ["low", "high"]}
    ],
    "settings": {"operation": "full_factorial"}
}
```

11.1.5 When to Use

Use full factorial when you have few factors (2–4) and can afford the number of runs. It provides complete information about all main effects and all interactions. It is the gold standard when resources allow.

Exponential growth

A full factorial with 7 two-level factors requires $2^7 = 128$ base runs. With 3 blocks, that is 384 runs. If each run takes 10 minutes, the experiment takes over 60 hours. Consider screening designs (PB or fractional factorial) when you have many factors.

11.2 Fractional Factorial

11.2.1 Overview

A fractional factorial tests a carefully chosen *subset* of the full factorial. The tool generates a 2^{k-p} Resolution III design, where k is the number of factors and p is determined automatically to minimize the number of runs while still estimating main effects.

11.2.2 Requirements

- Each factor must have **exactly 2 levels**.
- Requires `pyDOE3`.

11.2.3 Number of Runs

The number of base runs depends on the number of factors and the resolution. The tool computes the smallest k base factors such that k plus all interactions of those base factors covers all n factors:

Factors (n)	**Typical Base Runs**
3	4
4–7	8
8–15	16

11.2.4 Config Snippet

```
{
    "factors": [
        {"name": "A", "levels": ["low", "high"]},
        {"name": "B", "levels": ["low", "high"]},
        {"name": "C", "levels": ["low", "high"]},
        {"name": "D", "levels": ["low", "high"]},
        {"name": "E", "levels": ["low", "high"]}
    ],
    "settings": {"operation": "fractional_factorial"}
}
```

11.2.5 When to Use

Use fractional factorial when you have 4–15 two-level factors and want to estimate main effects with fewer runs than a full factorial. Be aware

that some higher-order interactions will be aliased (confounded) with main effects due to the resolution trade-off.

11.3 Plackett–Burman

11.3.1 Overview

Plackett–Burman (PB) designs are screening designs that can test $N - 1$ factors in N runs (where N is a multiple of 4). They are highly efficient for identifying which factors matter, but they cannot estimate interactions.

11.3.2 Requirements

- Each factor must have **exactly 2 levels**.
- Requires `pyDOE3`.

11.3.3 Number of Runs

The number of base runs is the smallest multiple of 4 that is $\geq n + 1$:

Factors	Base Runs
2–3	4
4–7	8
8–11	12
12–15	16
16–19	20

11.3.4 Config Snippet

```
{
    "factors": [
        {"name": "threads",            "levels": ["1", "4"]},
        {"name": "memory_block_size", "levels": ["1K",
    "16K"]},
        {"name": "memory_scope_type", "levels": ["global",
    "local"]},
        {"name": "memory_access_mode","levels": ["seq",
    "rnd"]},
        {"name": "memory_oper",        "levels": ["read",
    "write"]},
```

```
            {"name": "rand_type",          "levels": ["special",
        "uniform"]}
        ],
        "settings": {"operation": "plackett_burman"}
    }
```

11.3.5 When to Use

Use PB designs for initial screening when you have 4 or more factors and want to quickly identify the "vital few" that have the largest effects. Follow up with a more detailed design (full factorial or CCD) on the important factors only.

11.4 Latin Hypercube

11.4.1 Overview

Latin Hypercube Sampling (LHS) is a space-filling design that distributes sample points evenly across the factor space. Unlike factorial designs, it does not test specific level combinations but instead samples continuously within the factor ranges.

11.4.2 Requirements

- Each factor must have at least 2 levels.
- For continuous factors with 2 levels, LHS interpolates between the low and high values. For categorical factors, LHS bins the sample into discrete levels.
- Requires `pyDOE3`.

11.4.3 Number of Runs

The number of base runs is set by `lhs_samples` in the config. If set to 0 (the default), the tool uses $\max(10,\ 2 \times n_{\text{factors}})$. The "maximin" criterion is used to maximize the minimum distance between sample points.

11.4.4 Config Snippet

```
{
    "factors": [
        {"name": "temperature", "levels": ["150", "200"],
         "type": "continuous"},
        {"name": "pressure",    "levels": ["2", "10"],
         "type": "continuous"},
        {"name": "flow_rate",   "levels": ["5", "25"],
         "type": "continuous"}
    ],
    "settings": {
        "operation": "latin_hypercube",
        "lhs_samples": 20
    }
}
```

11.4.5 When to Use

Use LHS when you have continuous factors and want to explore the design space broadly without committing to a structured factorial design. LHS is particularly good for computer experiments (simulations) where there is no random error and you want to map the response surface with good coverage. It is also useful when the number of factors is large (10+) and factorial designs would require too many runs.

11.5 Central Composite Design (CCD)

11.5.1 Overview

A Central Composite Design adds star (axial) points and center points to a 2^k factorial design, enabling estimation of quadratic (curved) relationships. The tool generates a circumscribed CCD with orthogonal alpha values.

11.5.2 Requirements

- Each factor must have **exactly 2 numeric levels** (low and high).
- Levels must be parseable as floating-point numbers.
- Requires `pyDOE3`.

11.5.3 Number of Runs

For k factors: 2^k factorial points + $2k$ star points + center points (4 by default).

Factors	Factorial	Star	Center	Total
2	4	4	8	16
3	8	6	8	22
4	16	8	8	32
5	32	10	8	50

The CCD implementation uses `pyDOE3.ccdesign()` with `center=(4,4)`, `alpha="orthogonal"`, and `face="circumscribed"`.

11.5.4 Config Snippet

```
{
    "factors": [
        {"name": "temperature", "levels": ["150", "200"],
         "type": "continuous", "unit": "C"},
        {"name": "pressure",    "levels": ["2", "6"],
         "type": "continuous", "unit": "bar"}
    ],
    "settings": {"operation": "central_composite"}
}
```

11.5.5 When to Use

Use CCD when you want to model curvature in the response surface—for example, when you suspect there is an optimal temperature between your low and high settings, not at either extreme. CCD is the standard choice for response surface methodology (RSM) and is best used after a screening experiment has identified 2–5 important factors.

Star points extend beyond the factor range

In a circumscribed CCD, star points are located at a distance α from the center, which is *outside* the factorial range defined by your low and high levels. For example, if your temperature range is 150–200,

star points might be at 140 and 210. Make sure your experimental system can handle these extreme values.

11.6 Box–Behnken

11.6.1 Overview

A Box–Behnken design is a three-level design that avoids the extreme corner points of a full factorial. Each run uses either the center value or the low/high value for each factor, but never the corners where all factors are simultaneously at their extremes.

11.6.2 Requirements

- **At least 3 factors**.
- Each factor must have **exactly 2 numeric levels** (low and high).
- Levels must be parseable as floating-point numbers.
- Requires `pyDOE3`.

11.6.3 Number of Runs

The number of base runs depends on the number of factors and includes 3 center points:

Factors	Base Runs
3	15
4	27
5	46

11.6.4 Config Snippet

```
{
    "factors": [
        {"name": "temperature", "levels": ["150", "200"],
         "type": "continuous", "unit": "C"},
        {"name": "pressure",    "levels": ["2", "6"],
         "type": "continuous", "unit": "bar"},
        {"name": "catalyst",    "levels": ["0.5", "2.0"],
```

```
            "type": "continuous", "unit": "g/L"}
        ],
        "settings": {"operation": "box_behnken"}
    }
```

11.6.5 When to Use

Use Box–Behnken when you need to model curvature (like CCD) but want to avoid extreme corner combinations that might be physically dangerous, expensive, or infeasible. Box–Behnken requires at least 3 factors. It is generally more efficient than CCD for 3–5 factors because it avoids star points that extend beyond the factor range.

Chapter 12

Blocking and Randomization

12.1 How Blocking Works

The `block_count` setting in the config specifies how many times the entire base design is replicated. Each replication is called a "block." Blocking serves two purposes:

1. **Replication**: Running the same factor combinations multiple times provides a measure of experimental error and increases statistical power.

2. **Nuisance factor control**: If your experiment spans multiple days, machines, or batches, you can assign each block to a different day/machine/batch and later analyze whether the blocking factor introduced systematic variation.

12.1.1 Example: block_count = 1

With `block_count = 1` and a 2^3 full factorial, you get 8 runs, all in block 1:

```
run_id  block_id  A     B     C
-----------------------------------
1       1         low   low   A
2       1         low   low   B
3       1         low   high  A
...     1         ...   ...   ...
8       1         high  high  B
```

12.1.2 Example: block_count = 3

With `block_count = 3`, the same 8 combinations are replicated 3 times, giving 24 total runs:

```
run_id  block_id  A     B     C
------------------------------------
1       1         high  low   B
2       1         low   high  A
...
8       1         low   low   A
9       2         low   high  B
10      2         high  low   A
...
16      2         high  high  B
17      3         high  high  A
18      3         low   low   B
...
24      3         low   high  A
```

12.2 Within-Block Randomization

Runs are randomized *within* each block, not across blocks. This means:

- All runs in block 1 are executed first (in random order), then all runs in block 2, and so on.
- The run IDs are assigned sequentially after randomization (run 1 through N_{total}).
- The same seed always produces the same shuffled order within each block.

12.3 How –seed Controls Randomization

The seed is used to initialize a `random.Random` instance that shuffles runs within each block. For LHS designs, the seed additionally controls the LHS sampling via NumPy's random state.

- `--seed 42`: Deterministic, reproducible order.

- No `--seed`: The order depends on the design type. For full factorial, PB, FF, CCD, and BB, the base order from pyDOE3 (or itertools) is used without shuffling. For LHS, the NumPy default random state is used.

12.4 When to Use Blocking

- **Multi-day experiments**: Run one block per day. If day-to-day variation exists, it will be captured in the blocking structure.
- **Multiple machines**: Run one block per machine to isolate machine-specific effects.
- **Statistical power**: Even without nuisance factors, replication reduces the standard error of your effect estimates by $\sqrt{n}$.
- **Confidence**: With 3 blocks, each factor combination is tested 3 times. The summary statistics (mean, std) become much more reliable.

Blocks multiply run count

With b blocks and a base design of r runs, you execute $b \times r$ total runs. A 5-factor CCD with 3 blocks requires $50 \times 3 = 150$ runs. Plan your time budget accordingly.

Part V

Tutorials

Chapter 13

Tutorial — Database Performance Tuning

This tutorial walks through a realistic three-phase experiment to optimize MySQL database throughput by tuning five server configuration parameters.

13.1 The Problem

You are a database administrator and want to maximize query throughput (queries per second) for a web application workload. You suspect that five MySQL configuration knobs affect performance:

1. `innodb_buffer_pool_size`: 1 GB vs. 8 GB
2. `innodb_log_file_size`: 256 MB vs. 2 GB
3. `max_connections`: 100 vs. 500
4. `query_cache_size`: 0 vs. 256 MB
5. `thread_cache_size`: 8 vs. 64

You will use a three-phase approach: screen, refine, optimize.

13.2 Phase 1: Plackett–Burman Screening

With 5 factors, a PB design requires only 8 runs (the smallest multiple of 4 ≥ 6).

13.2.1 Configuration

```
{
    "metadata": {
        "name": "MySQL PB Screening",
        "description": "Identify which of 5 MySQL knobs
                        affect throughput most"
    },
    "factors": [
        {"name": "buffer_pool_size",
         "levels": ["1G", "8G"]},
        {"name": "log_file_size",
         "levels": ["256M", "2G"]},
        {"name": "max_connections",
         "levels": ["100", "500"]},
        {"name": "query_cache_size",
         "levels": ["0", "256M"]},
        {"name": "thread_cache_size",
         "levels": ["8", "64"]}
    ],
    "responses": [
        {"name": "qps", "optimize": "maximize",
         "unit": "queries/sec"}
    ],
    "settings": {
        "operation": "plackett_burman",
        "test_script": "./mysql_bench.sh",
        "out_directory": "results/phase1",
        "processed_directory": "results/phase1/analysis",
        "block_count": 2
    }
}
```

We use `block_count: 2` for replication, giving $8 \times 2 = 16$ total runs.

13.2.2 Commands

```
# Preview the design
$ doe info --config mysql_pb.json

# Generate the runner script
$ doe generate --config mysql_pb.json \
    --output phase1_runner.sh --seed 123

# Execute the experiments
```

```
$ bash phase1_runner.sh

# Analyze results
$ doe analyze --config mysql_pb.json
```

13.2.3 Interpreting the Results

```
=== Main Effects: qps ===
Factor                Effect     Std Error    % Contribution
-------------------------------------------------------------
buffer_pool_size     2450.0000    120.5000            48.2%
log_file_size        1380.0000     98.3000            27.2%
max_connections       720.0000     85.7000            14.2%
query_cache_size      340.0000     79.2000             6.7%
thread_cache_size     190.0000     76.1000             3.7%
```

The Pareto chart shows that `buffer_pool_size` and `log_file_size` together account for 75.4% of the total effect. These are the "vital few" factors to investigate further.

13.3 Phase 2: Full Factorial on Top 2 Factors

Now focus on the two most impactful factors with a 2^2 full factorial.

13.3.1 Configuration

```
{
    "metadata": {
        "name": "MySQL Full Factorial",
        "description": "Detailed study of buffer pool and
                        log file size"
    },
    "factors": [
        {"name": "buffer_pool_size",
         "levels": ["1G", "8G"]},
        {"name": "log_file_size",
         "levels": ["256M", "2G"]}
    ],
    "fixed_factors": {
        "max_connections": "300",
```

```
            "query_cache_size": "128M",
            "thread_cache_size": "32"
        },
        "responses": [
            {"name": "qps", "optimize": "maximize",
             "unit": "queries/sec"}
        ],
        "settings": {
            "operation": "full_factorial",
            "test_script": "./mysql_bench.sh",
            "out_directory": "results/phase2",
            "processed_directory": "results/phase2/analysis",
            "block_count": 3
        }
    }
```

Notice that the three non-critical factors are now held constant as `fixed_factors` at their midpoints.

13.3.2 Commands and Analysis

```
$ doe generate --config mysql_ff.json \
    --output phase2_runner.sh --seed 456
Generated 12 runs -> phase2_runner.sh

$ bash phase2_runner.sh

$ doe analyze --config mysql_ff.json

=== Interaction Effects: qps ===
Factor A             Factor B             Interaction
                                                    %
    Contribution
----------------------------------------------------------------------
buffer_pool_size     log_file_size             850.0000
       100.0%
```

The significant interaction tells us that the effect of increasing buffer pool size is larger when the log file size is also large. This synergy suggests both should be set high together.

13.4 Phase 3: CCD Optimization

For the final phase, use a Central Composite Design with numeric levels to find the precise optimum.

13.4.1 Configuration

```
{
    "metadata": {
        "name": "MySQL CCD Optimization",
        "description": "Find optimal buffer pool and
                        log file size settings"
    },
    "factors": [
        {"name": "buffer_pool_size_gb",
         "levels": ["2", "8"], "type": "continuous",
         "unit": "GB"},
        {"name": "log_file_size_mb",
         "levels": ["512", "2048"], "type": "continuous",
         "unit": "MB"}
    ],
    "fixed_factors": {
        "max_connections": "300",
        "query_cache_size": "128M",
        "thread_cache_size": "32"
    },
    "responses": [
        {"name": "qps", "optimize": "maximize",
         "unit": "queries/sec"}
    ],
    "settings": {
        "operation": "central_composite",
        "test_script": "./mysql_bench.sh",
        "out_directory": "results/phase3",
        "processed_directory": "results/phase3/analysis"
    }
}
```

13.4.2 Commands

```
$ doe generate --config mysql_ccd.json \
```

```
    --output phase3_runner.sh --seed 789
Generated 16 runs -> phase3_runner.sh

$ bash phase3_runner.sh

$ doe optimize --config mysql_ccd.json

=== Optimization: qps ===
Direction: maximize

Best observed run: #12
  buffer_pool_size_gb = 8
  log_file_size_mb = 2048
  Value: 14520.0

RSM Model (linear, R^2 = 0.91):
  Coefficients:
    intercept:  +11250.0000
    buffer_pool_size_gb:  +1580.0000
    log_file_size_mb:  +920.0000
  Predicted optimum:
    buffer_pool_size_gb = 8
    log_file_size_mb = 2048
    Predicted value: 14350.0000

Factor importance:
  1. buffer_pool_size_gb  (effect: 3160.0,
     contribution: 63.2%)
  2. log_file_size_mb  (effect: 1840.0,
     contribution: 36.8%)
```

13.4.3 Conclusion

The three-phase approach took us from 5 unknown factors to a clear recommendation: set `buffer_pool_size` to 8 GB and `log_file_size` to 2 GB for maximum throughput. The RSM model with $R^2 = 0.91$ gives us high confidence in these settings.

Finally, generate a report to share with the team:

```
$ doe report --config mysql_ccd.json \
    --output mysql_optimization_report.html
HTML report written to: mysql_optimization_report.html
```

Chapter 14

Tutorial — Chemical Reactor Optimization

This tutorial uses the included `reactor_config.json` example to optimize a chemical batch reactor with three continuous factors and three responses.

14.1 The Problem

You are optimizing a batch chemical reactor. Three process parameters can be adjusted:

1. **Temperature**: 150–200 °C
2. **Pressure**: 2–6 bar
3. **Catalyst concentration**: 0.5–2.0 g/L

Two parameters are held constant: reaction time (60 min) and stirring speed (300 rpm).

Three responses are measured:

- **Yield** (%): maximize
- **Purity** (%): maximize
- **Cost** (USD): minimize

14.2 The Configuration

```
{
    "metadata": {
        "name": "Chemical Reactor Optimization",
        "description": "Box-Behnken design to optimize
```

```
                yield and purity of a batch reactor process"
        },
        "factors": [
            {"name": "temperature", "levels": ["150", "200"],
             "type": "continuous", "unit": "C",
             "description": "Reactor temperature"},
            {"name": "pressure",     "levels": ["2", "6"],
             "type": "continuous", "unit": "bar",
             "description": "Operating pressure"},
            {"name": "catalyst",     "levels": ["0.5", "2.0"],
             "type": "continuous", "unit": "g/L",
             "description": "Catalyst concentration"}
        ],
        "fixed_factors": {
            "reaction_time": "60",
            "stirring_speed": "300"
        },
        "responses": [
            {"name": "yield",  "optimize": "maximize",
             "unit": "%"},
            {"name": "purity", "optimize": "maximize",
             "unit": "%"},
            {"name": "cost",   "optimize": "minimize",
             "unit": "USD"}
        ],
        "runner": {"arg_style": "double-dash"},
        "settings": {
            "block_count": 1,
            "test_script": "examples/reactor_sim.sh",
            "operation": "box_behnken",
            "processed_directory": "results/reactor/analysis",
            "out_directory": "results/reactor"
        }
    }
```

14.3 Why Box–Behnken?

Box–Behnken is ideal here because:

- We have exactly 3 continuous factors (BB requires at least 3).
- We want to model curvature but do not want extreme corner points (e.g., running at 200 °C *and* 6 bar *and* 2.0 g/L catalyst simultaneously

might be dangerous).

- BB generates only 15 runs (vs. 22 for CCD with 3 factors), saving time and materials.

14.4 Step 1: Preview the Design

```
$ doe info --config examples/reactor_config.json
Plan      : Chemical Reactor Optimization
Desc      : Box-Behnken design to optimize yield and
            purity of a batch reactor process
Operation : box_behnken
Factors   : temperature, pressure, catalyst
Base runs : 15
Blocks    : 1
Total runs: 15
Responses : yield [maximize] (%), purity [maximize] (%),
            cost [minimize] (USD)
Fixed     : reaction_time=60, stirring_speed=300

run_id  block_id  temperature  pressure  catalyst
-------------------------------------------------------
1       1         150          2         1.25
2       1         200          2         1.25
3       1         150          6         1.25
4       1         200          6         1.25
5       1         150          4         0.5
6       1         200          4         0.5
7       1         150          4         2
8       1         200          4         2
9       1         175          2         0.5
10      1         175          6         0.5
11      1         175          2         2
12      1         175          6         2
13      1         175          4         1.25
14      1         175          4         1.25
15      1         175          4         1.25
```

Notice the structure: runs 1–4 vary temperature and pressure while holding catalyst at center (1.25). Runs 5–8 vary temperature and catalyst while holding pressure at center (4). Runs 9–12 vary pressure and catalyst while holding temperature at center (175). Runs 13–15 are center points (all

factors at their midpoints).

14.5 Step 2: Generate and Execute

```
$ doe generate --config examples/reactor_config.json \
    --output reactor_runner.sh --seed 100
Generated 15 runs -> reactor_runner.sh

$ bash reactor_runner.sh
Starting 15 experiment runs...

=== Run 1 / 15 (Block 1) ===
  temperature = 175
  pressure = 4
  catalyst = 1.25
...
All runs complete. Results in: results/reactor
```

Each run produces a JSON file with three response keys:

```
{"yield": 82.4, "purity": 97.1, "cost": 185.20}
```

14.6 Step 3: Analyze

```
$ doe analyze --config examples/reactor_config.json
```

The output now shows three separate analysis sections—one per response:

```
=== Main Effects: yield ===
Factor                Effect     Std Error    % Contribution
-------------------------------------------------------------
temperature            8.5000      0.9200            45.7%
pressure               6.2000      0.8100            33.3%
catalyst               3.9000      0.7500            21.0%

=== Main Effects: purity ===
Factor                Effect     Std Error    % Contribution
-------------------------------------------------------------
catalyst               2.8000      0.3100            48.3%
```

```
temperature           1.9000      0.2800           32.8%
pressure              1.1000      0.2500           19.0%

=== Main Effects: cost ===
Factor                Effect    Std Error   % Contribution
---------------------------------------------------------------
temperature          25.0000      2.1000           50.0%
catalyst             15.5000      1.8000           31.0%
pressure              9.5000      1.5000           19.0%
```

14.7 Step 4: Optimize

```
$ doe optimize --config examples/reactor_config.json

=== Optimization: yield ===
Direction: maximize
Best observed run: #4
  temperature = 200
  pressure = 6
  catalyst = 1.25
  Value: 94.5
...

=== Optimization: purity ===
Direction: maximize
Best observed run: #12
  temperature = 175
  pressure = 6
  catalyst = 2
  Value: 99.2
...

=== Optimization: cost ===
Direction: minimize
Best observed run: #5
  temperature = 150
  pressure = 4
  catalyst = 0.5
  Value: 125.30
...
```

14.8 Multi-Response Trade-offs

The optimization reveals a classic trade-off:

- **Maximum yield** requires high temperature and high pressure.
- **Maximum purity** requires moderate temperature with high catalyst.
- **Minimum cost** requires low temperature and low catalyst.

You cannot simultaneously maximize yield, maximize purity, and minimize cost. The optimize command shows the best settings for each response independently. In practice, you would choose a compromise based on your priorities—for example, set temperature to 175 (moderate), pressure to 6 (high), and catalyst to 1.5 (medium-high) to get good yield and purity at an acceptable cost.

Using –response for focused optimization

To focus on a single response:

```
$ doe optimize --config examples/reactor_config.json \
    --response yield
```

This prints only the yield optimization, which is cleaner when you have decided which response to prioritize.

14.9 Step 5: Generate a Report

```
$ doe report --config examples/reactor_config.json \
    --output reactor_report.html
HTML report written to: reactor_report.html
```

The HTML report contains collapsible sections for each response, with embedded Pareto charts and main effects plots. The design matrix section (collapsed by default) shows all 15 runs with their Box–Behnken factor values.

Appendix A

Error Messages Reference

This appendix catalogs every error message the tool can produce, with the cause and the fix.

A.1 Configuration Validation Errors

These errors are raised by `config.py` during `load_config()`.

Error Message	Cause	Fix
`Unsupported operation '{op}'. Choose from: [box_behnken, central_composite, fractional_factorial, full_factorial, latin_hypercube, plackett_burman]`	The `settings.operation` value is not one of the six supported design types.	Set `ope` values l
`At least one factor is required.`	The `factors` array is empty or missing.	Add at l and at le
`Factor names must be unique, got: […]`	Two or more factors have the same `name`.	Give ea
`block_count must be >= 1, got {n}`	The `block_count` is set to 0 or a negative number.	Set `blo`

Error Message	Cause	Fix
`Plackett-Burman requires exactly 2 levels per factor, but factor '{name}' has {n}: […]`	A PB design was requested but a factor has more (or fewer) than 2 levels.	Ensure every els, or use a
`Fractional factorial requires exactly 2 levels per factor, but factor '{name}' has {n}: […]`	A fractional factorial was requested but a factor does not have exactly 2 levels.	Set each fac
`Central composite requires exactly 2 levels (low, high) per factor, but factor '{name}' has {n}: […]`	A CCD was requested but a factor does not have exactly 2 levels.	Set each fac levels.
`Central composite requires numeric levels, but factor '{name}' has non-numeric levels: […]`	A CCD was requested but a factor's levels cannot be parsed as floats.	Use numeri `"200"]`) for
`Box-Behnken requires at least 3 factors, but only {n} were provided.`	A BB design was requested with fewer than 3 factors.	Add more f design type
`Box-Behnken requires exactly 2 levels (low, high) per factor, but factor '{name}' has {n}: […]`	A BB design was requested but a factor does not have exactly 2 levels.	Set each fac levels.
`Box-Behnken requires numeric levels, but factor '{name}' has non-numeric levels: […]`	A BB design was requested but a factor's levels are not numeric.	Use numeric els.
`Response names must be unique, got: […]`	Two or more responses have the same `name`.	Give each re
`Response '{name}' has invalid optimize='{val}'. Choose from: [maximize, minimize]`	A response's `optimize` field is not `maximize` or `minimize`.	Set `optimi` or `"minimi`

Error Message	Cause	Fix
`runner.arg_style '{val}' is invalid. Choose from: [double-dash, env, positional]`	The `runner.arg_style` field is not one of the three valid values.	Set `arg` `"env"`,
`Factor must have a name and at least 2 levels: {item}`	A factor dict is missing `name` or has fewer than 2 levels.	Provide in `leve`
`Factor must have a name and at least one level: {item}`	A legacy-format factor array is empty or has only one element.	Provide level va

A.2 Runtime Errors

Error Message	Cause	Fix
`pyDOE3 is required for {design} designs. Install it with: pip install pyDOE3`	The `pyDOE3` package is not installed but is needed for the requested design type.	Run `pi`
`Missing result files for run IDs: […]. Expected in: {dir}`	The `analyze`, `optimize`, or `report` command could not find one or more result JSON files.	Ensure pleted s put dire by delet executin
`Warning: response '{name}' missing in result files for run IDs: […]`	Some result JSON files exist but do not contain the expected response key.	Check t correct
`Warning: test_script '{path}' does not exist.`	During `generate` (strict mode), the test script path does not exist.	Create path in t not a fa

Appendix B

Output File Formats

B.1 Result JSON Files (Per-Run)

Each experiment run produces a file named `run_{N}.json` in the output directory. The file is a JSON object whose keys must match the response names in the configuration:

```
{"yield": 87.3, "purity": 99.1, "cost": 142.50}
```

For a single response named `"response"`:

```
{"response": 87.3}
```

The naming convention is `run_{run_id}.json` where `run_id` is the sequential integer assigned during design generation.

B.2 CSV Export Format

The `--csv DIR` flag on the `analyze` command creates two files per response:

B.2.1 main_effects_{response}.csv

factor_name	main_effect	std_error	pct_contribution	ci_low	ci_h
temperature	12.45	1.234	52.1	9.82	15.0
pressure	7.82	0.987	32.7	5.71	9.93

All values are floating-point numbers. `ci_low` and `ci_high` are 0.0 if confidence intervals could not be computed.

B.2.2 summary_stats_{response}.csv

factor	level	n	mean	std	min	max
temperature	high	4	91.2	3.45	87.3	95.1
temperature	low	4	78.75	2.89	75.2	82.3

B.3 HTML Report Structure

The HTML report is a single file with embedded CSS (no external stylesheets) and base64-encoded PNG images. The structure uses semantic HTML5:

- `<header>`: Title, description, timestamp.
- `<details>` sections for Design Summary, Results per response, and Design Matrix.
- `<table>` elements with CSS classes `info-table` and `data-table`.
- `<img>` tags with `…` source URIs.
- `<footer>`: "Generated by DOE Helper Tool."

B.4 Plot PNG Files

Two PNG files are generated per response in the `processed_directory`:

File Name	Content
`pareto_{response}.png`	Horizontal bar chart of absolute main effects with cu
`main_effects_{response}.png`	Subplot grid showing mean response per level per fac

Both are rendered at 150 DPI. Special characters in response names (`/`, spaces) are replaced with underscores in file names.

Appendix C

Configuration Examples

C.1 Minimal Configuration

The smallest valid configuration: two factors, full factorial, no metadata:

```
{
    "factors": [
        {"name": "A", "levels": ["0", "1"]},
        {"name": "B", "levels": ["0", "1"]}
    ],
    "settings": {
        "operation": "full_factorial",
        "test_script": "./test.sh",
        "out_directory": "results"
    }
}
```

This produces 4 runs. Responses default to a single response named `"response"` with `optimize="maximize"`. The runner section defaults to `arg_style="double-dash"`.

C.2 Full-Featured Configuration

Every field specified:

```
{
    "metadata": {
        "name": "Comprehensive Experiment",
        "description": "Testing all configuration fields"
```

```
    },
    "factors": [
        {"name": "temperature", "levels": ["150", "200"],
         "type": "continuous", "unit": "C",
         "description": "Process temperature"},
        {"name": "pressure", "levels": ["2", "6"],
         "type": "continuous", "unit": "bar",
         "description": "Operating pressure"},
        {"name": "catalyst", "levels": ["0.5", "2.0"],
         "type": "continuous", "unit": "g/L",
         "description": "Catalyst concentration"}
    ],
    "fixed_factors": {
        "reaction_time": "60",
        "stirring_speed": "300"
    },
    "responses": [
        {"name": "yield", "optimize": "maximize",
         "unit": "%",
         "description": "Product yield percentage"},
        {"name": "purity", "optimize": "maximize",
         "unit": "%",
         "description": "Product purity"},
        {"name": "cost", "optimize": "minimize",
         "unit": "USD",
         "description": "Batch production cost"}
    ],
    "runner": {
        "arg_style": "double-dash",
        "result_file": "json"
    },
    "settings": {
        "block_count": 2,
        "test_script": "./run_experiment.sh",
        "operation": "box_behnken",
        "processed_directory": "output/analysis",
        "out_directory": "output/results",
        "lhs_samples": 0
    }
}
```

C.3 Multi-Response Configuration

An experiment measuring three responses simultaneously:

```
{
    "metadata": {
        "name": "Multi-Response Study"
    },
    "factors": [
        {"name": "speed",     "levels": ["100", "500"]},
        {"name": "feed",      "levels": ["0.1", "0.5"]},
        {"name": "depth",     "levels": ["1", "5"]}
    ],
    "responses": [
        {"name": "surface_roughness",
         "optimize": "minimize", "unit": "um"},
        {"name": "material_removal_rate",
         "optimize": "maximize", "unit": "mm3/min"},
        {"name": "tool_wear",
         "optimize": "minimize", "unit": "mm"}
    ],
    "settings": {
        "operation": "box_behnken",
        "test_script": "./machining_test.sh",
        "out_directory": "results/machining"
    }
}
```

Your test script must produce JSON with all three keys:

```
{
    "surface_roughness": 1.23,
    "material_removal_rate": 45.6,
    "tool_wear": 0.05
}
```

C.4 Legacy Format Configuration

Using the older `static_settings` and array-format factors:

```
{
    "factors": [
        ["temperature", "low", "high"],
```

```
            ["pressure", "low", "high"],
            ["catalyst", "A", "B"]
        ],
        "static_settings": [
            "--duration=60",
            "--warmup-time=3"
        ],
        "settings": {
            "operation": "full_factorial",
            "test_script": "./test.sh",
            "out_directory": "results"
        }
    }
```

This is equivalent to the modern format with:

- Factors as dictionaries with `type="categorical"`.
- `fixed_factors: {"duration": "60", "warmup-time": "3"}`.
- Default single response named `"response"`.

The tool automatically converts the legacy format during loading, so all commands work identically.

Index of Commands

`enerate`	Generate experiment runner script from config. See Chapter 6.
`nalyze`	Compute main effects, interactions, and summary statistics. See Chapter 7.
`ptimize`	Find optimal factor settings with RSM. See Chapter 8.
`nfo`	Preview design without generating files. See Chapter 9.
`eport`	Generate self-contained HTML report. See Chapter 10.